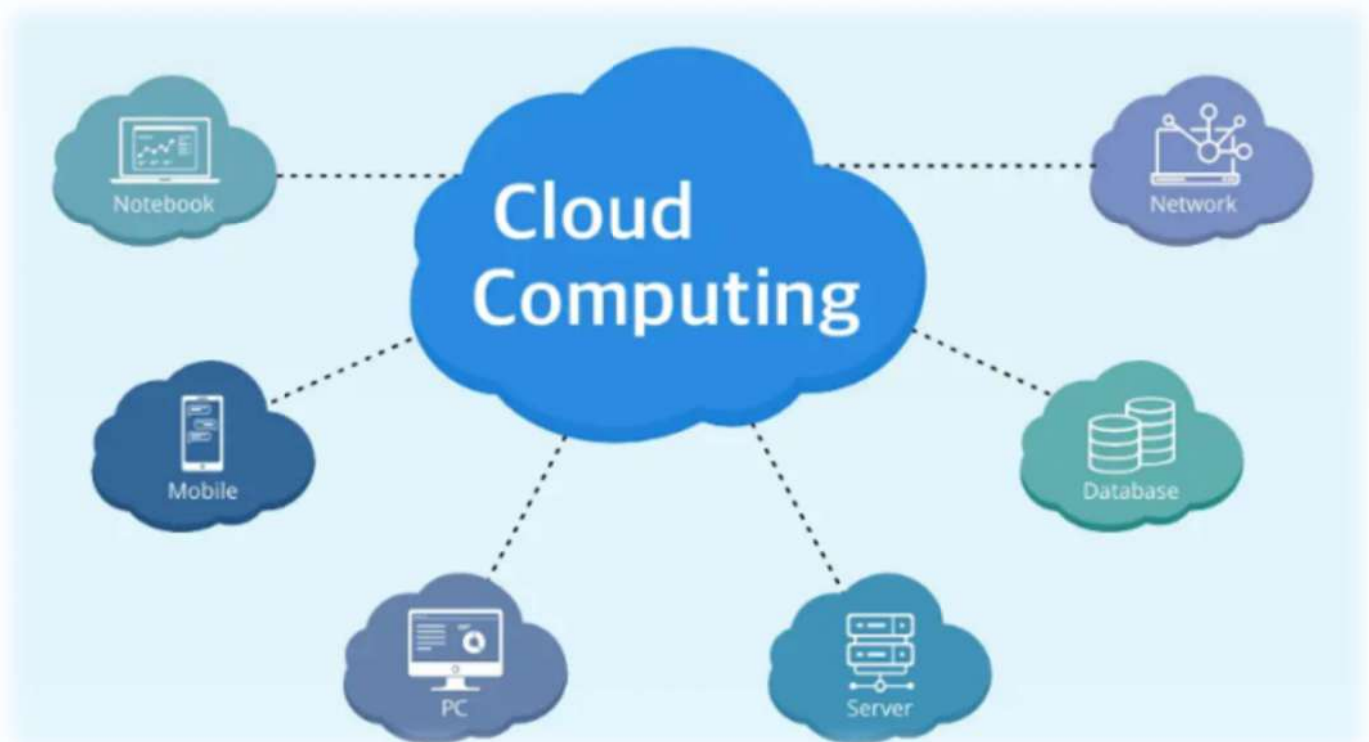


COURSE OBJECTIVE_04

Public Cloud Platform-Architectures and Programming:

- ✚ Google App Engine (GAE)
- ✚ Amazon Web Services (AWS)
- ✚ Microsoft Windows Azure
- ✚ Service Oriented Architecture: REST
- ✚ Publish Subscribe Model



Public Cloud Platform-Architectures and Programming

A public cloud platform is a third-party managed environment that delivers IT resources—such as servers, storage, and applications—to multiple customers over the internet using a multi-tenant architecture. Users access services via web browsers or APIs and typically pay on a pay-as-you-go basis.

Architecture Components:

Public cloud architecture consists of front-end and back-end components connected via the internet.

Key Architectural Characteristics

Multi-tenancy: A fundamental characteristic where multiple organizations share the same physical infrastructure, but their data and workloads are logically isolated for security and privacy.

Virtualization: This technology divides physical resources into virtual machines (VMs), containers, or server-less instances, which are then allocated to customers as needed.

Scalability and Elasticity: Resources can be rapidly scaled up or down automatically based on demand, ensuring applications remain responsive during traffic spikes.

High Availability and Redundancy: Providers use globally distributed data centers and data replication to ensure continuous availability and protect against outages or data loss.

Programming and Service Models

Public cloud services are broadly categorized into three main "as a Service" models (XaaS), which dictate the level of control and responsibility shared between the provider and the user.

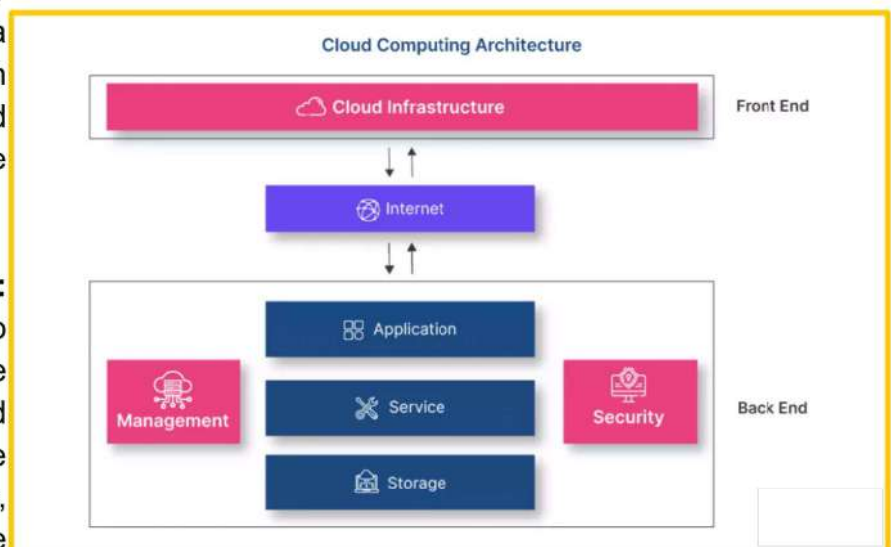
Infrastructure as a Service (IaaS):

Provides on-demand access to fundamental IT infrastructure like virtual machines, storage, and networking. Users manage the operating systems and applications, while the provider manages the physical hardware.

Examples: Amazon EC2, Azure Virtual Machines, Google Compute Engine.

Platform as a Service (PaaS): Offers a platform and tools for developers to build, test, deploy, and manage applications without managing the underlying infrastructure.

Examples: AWS Elastic Beanstalk, Azure App Service, Google App Engine.



Software as a Service (SaaS): Delivers fully functional software applications over the internet on a subscription basis. The provider manages everything from the infrastructure to the application itself.

Examples: Google Workspace, Microsoft 365, Salesforce.

Serverless Computing (FaaS): An execution model where developers run application code (functions) without managing any servers or infrastructure. The cloud provider handles all the underlying resource management and scaling.

Examples: AWS Lambda, Azure Functions, Google Cloud Functions.

Major Providers

The public cloud market is dominated by a few major players, each offering a comprehensive suite of services:

Amazon Web Services (AWS)

Microsoft Azure

Google Cloud Platform (GCP).

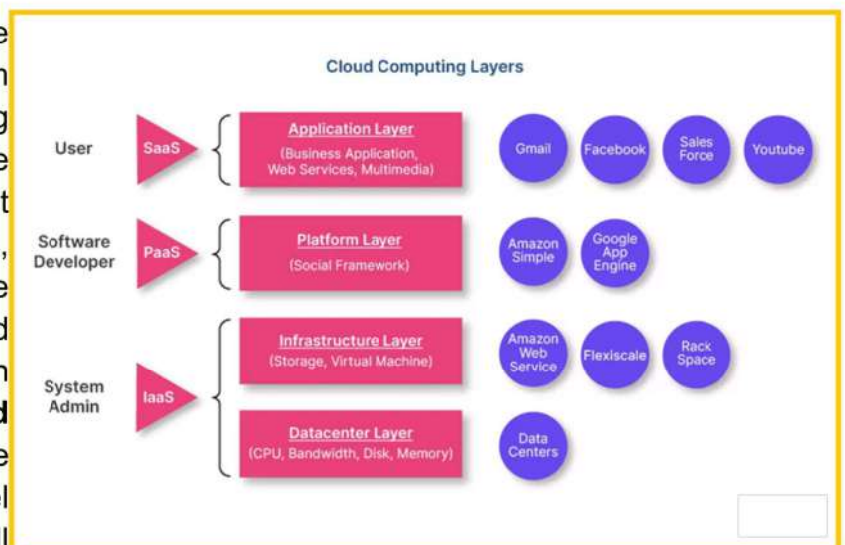
Front End: This is the client side, including user interfaces (web portals, mobile apps, CLIs) that allow users to access and manage services. It abstracts the underlying complexity of the cloud infrastructure.

Back End: Managed by the cloud service provider (CSP), this is the core infrastructure where data is stored and processed. Components include physical servers, storage, networking devices, virtualization software (hypervisors), runtime environments, and management and security software.

Network: The internet or dedicated connections act as the bridge, facilitating communication and data exchange between the front end and the back end

How does it work?

Cloud architecture is all about the arrangement of the materials that form the environment for cloud computing platforms. It combines SOA (Service Oriented Architecture) and EDA (Event Driven Architecture). In the back end, cloud service providers provide resources, services, data storage, and applications. Using a network, data can be transported between **front-end and back-end cloud architecture** components. The service model determines how the communication will



take place. This means that cloud computing enables users to utilize remote computing resources on demand, including servers, storage, databases, networking, software, analytics, and intelligence through the internet rather than possessing such amenities at their locality

Conclusion

Modern technological infrastructure is primarily built on cloud computing architecture, which revolves around business activities and how people relate to digital resources. Hardware, software, networking, and services form this structured architecture of cloud computing technology.

The triumvirate of public, private, and hybrid cloud architectures offers organizations a spectrum of choices, exemplified by industry giants like AWS, Azure Stack, and Google Anthos. Each type caters to distinct needs, providing scalability, security, and flexibility.

Finally, cloud computing architecture surpasses conventional approaches that provide an agile, economical, and scalable platform. It acts as a chain in business chain adaption to the new age of improved efficiency, innovation, and a secure future.

Google App Engine (GAE)

Imagine a platform where you can focus solely on coding, free from the complexities of infrastructure management. Google App Engine in Cloud Computing offers exactly that, a fully managed serverless environment designed to simplify the development and hosting of web applications. Sanctioning a variety of programming languages and frameworks, it automatically handles server provisioning, scaling, and traffic management. Whether you're working on a small project or scaling a large enterprise solution, Google App Engine provides the tools and flexibility to grow with your needs, ensuring your success.

What is Google App Engine?

Google App Engine in Cloud Computing enables web developers and enterprises to build, deploy, and host scalable, high-performance applications within a fully managed environment. It eliminates the need for infrastructure provisioning or management, making application development more efficient and streamlined.

As a fully managed and serverless application development platform, GAE handles the entire process of uploading and running code on Google Cloud. It automatically provisions the necessary infrastructure based on the CPU and memory requirements specified by the developer.

GAE supports multiple programming languages and custom containers, providing development kits and runtimes for development, testing, and debugging. It also grants a comprehensive suite of developer tools to streamline app development, testing, debugging, deployment, and performance monitoring.

How is the Google App Engine Used?

Google App Engine in Cloud Computing is a serverless platform that hosts and allows developers to build and deploy web applications. Developers or users can create an account in Google App Engine to set up a Software Development Kit (SDK), to write the source code of applications easily.

It is also used to build scalable back end mobile applications. These are then used to adapt workloads as needed. Google App Engine can also be used for application testing where users can route traffic to different application versions.

The Architecture of Google App Engine

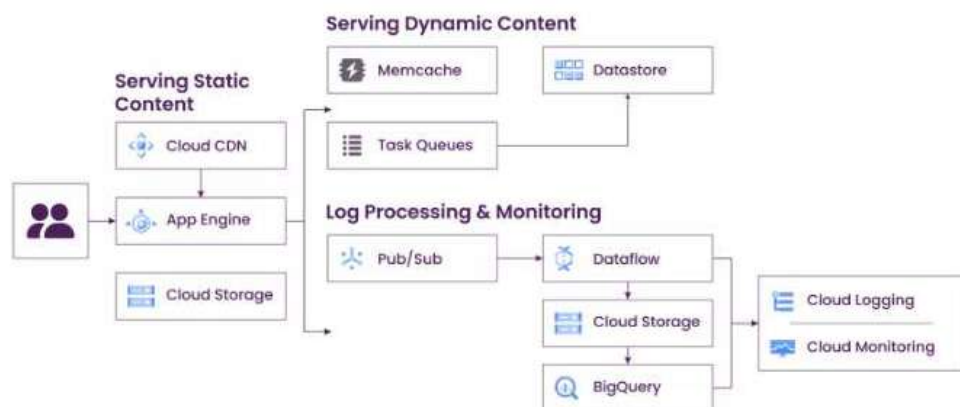
Google App Engine's architecture in Cloud Computing is both scalable and robust, designed to cater to a diverse range of applications and services. Here's a concise breakdown of its structure below:

Architecture of Google App Engine

1) Datastore: Serving as the central data management system in Cloud Computing, Google App Engine's Datastore is a NoSQL database renowned for its scalability. What sets it apart is its dynamic nature, adapting in real-time to the demands of the application. Whether it's a minor data retrieval or a massive data influx, the datastore scales on-the-fly, ensuring that data remains consistently accessible and safeguarded against potential threats.

2) Task Queues: In any application, there exist tasks that don't necessitate immediate user feedback. Google App Engine's Task queues are designed to manage such background operations. By queuing these tasks, they're executed asynchronously, optimising application performance and ensuring users aren't bogged down with processing delays.

Architecture of Google App Engine



3) Memcache: As a rapid-access in-memory caching system, Memcache plays a pivotal role in enhancing data retrieval speeds. Especially beneficial for frequently queried data, it acts as a buffer, reducing the datastore's workload. This not only ensures quicker response times but also contributes to the longevity and efficiency of the main Datastore.

4) Blobstore: In today's digital age, applications often deal with voluminous data, be it high-definition images, videos, or other large files. The Blobstore is Google App Engine's dedicated solution for such requirements. By efficiently managing and storing these large objects, it ensures that the primary datastore isn't overwhelmed, maintaining a harmonious data ecosystem.

5) Automatic Scaling: One of Google App Engine's crowning features, Automatic Scaling, epitomises proactive resource management. By continually monitoring application traffic and user requests, it dynamically scales resources. This ensures optimal performance even during unexpected traffic surges, eliminating the need for manual adjustments and guaranteeing a consistently smooth user experience.

6) Integrated Services: Google App Engine isn't an isolated entity but a cog in the vast machinery of Google Cloud Computing services. Its ability to seamlessly mesh with other services, from Data Analytics platforms to state-of-the-art Machine Learning tools, transforms it from a mere hosting platform to a comprehensive, integrated Cloud solution. This interoperability enhances the capabilities of applications hosted on Google App Engine, giving Developers a richer toolset to work with.

Features of App Engine

Google Cloud Platform provides two options for utilising App Engine: The Standard Environment and the Flexible Environment.

Standard Environment

The Standard Environment operates on Google's managed infrastructure and is compatible with languages such as Python, Go, and Node.js. It is created for tasks with high levels of demand and extensive sets of data. Some of the main characteristics are:

- a) Persistent Cloud Storage that includes querying, sorting, and transaction functionalities
- b) Automated adjustment of capacity and distribution of workload
- c) Task queues that handle background work in an asynchronous manner
- d) Planned activities for recurring or time-based occasions
- e) Collaboration with additional services and interfaces in Google Cloud

Flexible Environment

The Flexible Environment, built on Google Compute Engine, offers developers increased flexibility. It provides custom runtimes with Docker, extends request and response timeouts, and allows for the installation of custom software. Some important characteristics are:

- a) Customised Infrastructure: Users have the option to use custom libraries, SSH for debugging, and deploy Docker containers on virtual machines in Compute Engine.
- b) Open-source Community: Backs a variety of open-source tools and frameworks
- c) Native Feature Support: Support for native features includes microservices, authorisation, SQL and NoSQL databases, traffic splitting, and logging.
- d) Performance: Provides various CPU and memory setups for the best possible performance.

Google App Engine Apps

A Google App Engine (GAE) application consists of a primary container that includes a minimum of one service, version, and application resource.

What are the Benefits and Challenges of Using Google App Engine?

In this section, we are going to discuss about the several benefits and challenges of using Google App Engine. These are as follows:

Benefits of Using Google App Engine

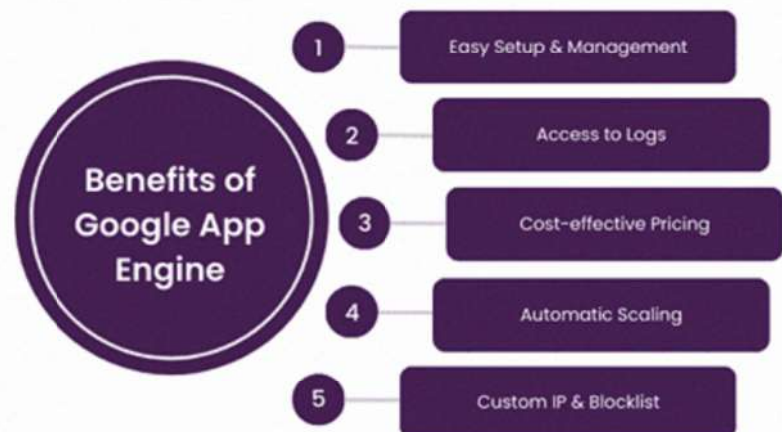
a) Easy Setup & Management: One of the key advantages of Google App Engine in Cloud Computing is its ease of setup. It is also easy to manage, which makes it more attractive to users. Developers can write code without needing help from core IT professionals. It also allows users to build back-end infrastructure. The built-in APIs in Google App Engine enable users to develop various types of applications.

b) Access to Logs: It also has the access to all the application logs which facilitates debugging and monitoring in production.

c) Cost-effective Pricing: The pay-per-use pricing of Google App Engine makes it cost-effective and therefore users flock towards it. It has several resources which the users can use to monitor their resources and keep track of their bills in the dashboard.

d) Automatic Scaling: It can automatically scale as the workloads fluctuate for the users. It can automatically add and remove application resources as per the demand.

e) Custom IP & Blocklist: It can specify a range of Internet Protocol addresses (IP) which are acceptable. Moreover, it also has the bandwidth to allow users to prepare a specific list of networks and resources and prepare blocklist for certain IP addresses.



Challenges of Using Google App Engine

Google App Engine has some drawbacks as well. Let's discuss some of the challenges of using Google App Engine:

a) Dependency on Google: Because of its back-end infrastructure it is sometimes difficult for users to fix their problems on their own. They have to depend on Google to fix those problems, which can sometimes be time-consuming.

b) Performance Limits: Google App Engine has performance limits. This means that all CPU-intensive operations can be a little bit expensive and slow when using Google App Engine. The reason for this limitation can be that one physical server may serve numerous separate and related App Engine users at the same time, who need to share the CPU, as explained in the Cloud Computing Basics PDF.

c) Limited Filesystem Access: GAE only provides limited read-only access to Google App Engine filesystem to Developers.

d) Java app Limitations: It limits Java apps. In Google App Engine Java apps are not able to create new threads and only certain sections of Java runtime environment can be implemented.

Why Choose Google App Engine?

Google App Engine in Cloud Computing possesses a list of benefits over other services, which make it a standard choice for enterprises. Some of these benefits are as follows:

1) Cost-effective

One of the most compelling reasons to opt for Google App Engine is its cost-efficiency. Unlike traditional hosting, where overheads can rack up quickly, this tool's fully managed nature means there's no need to invest in system administration. In this Cloud Computing model, you pay strictly for the resources you consume.

This allows businesses, from startups to large enterprises, to scale their operations without the dread of unexpected costs. The adaptability of adjusting your usage according to the demand ensures that you're always in control of your expenses. For a deeper understanding, refer to the Google Cloud Platform Fundamentals PDF for detailed insights into the pricing structure and benefits.

Why Choose Google App Engine?



2) Productivity Boost

Time is of the essence in the Cloud Computing domain. It is attuned to this reality. By taking on the responsibility of infrastructure-related tasks, it liberates Developers to zero in on what truly matters: crafting exceptional code and refining their application. Without the distractions of backend management, Developers experience a tangible uptick in their productivity, ensuring that projects are delivered faster, and innovations reach the market promptly.

3) Security

Nowadays cyber threats are rife, thus security isn't just a feature, it's a necessity. The Google App Engine stands tall on the foundation of Google's globally recognised secure infrastructure. But it doesn't just rest there; it continually fortifies its defences. Google App Engine uses its built-in security protocols and a commitment to meet the strictest compliance standards to ensure that your applications are ensconced in a digital fortress, safeguarded against potential threats.

4) Versatility

Google App Engine's versatility shines through in its ability to support several projects. Whether you're aiming to launch a minimalist web application or architecting a labyrinthine system teeming with features, it is up to the task. From serving static web pages to managing backend operations for mobile applications bustling with high traffic, it proves time and again that it's a multifaceted Cloud Computing solution equipped to handle the diverse needs of the digital age.

Conclusion

The prominence of Google App Engine in Cloud Computing cannot be understated. It serves as a beacon of advanced design and user-friendliness, catering to both Developers and enterprises seamlessly. It stands out as the go-to choice for those seeking a dependable and secure Cloud infrastructure. When one aligns with Google App Engine, they're not only embracing the pinnacle of Cloud technology but also ensuring their applications consistently deliver top-tier performance.

Amazon Web Services (AWS)

Amazon Web Services (AWS) was started in 2006 to help companies avoid the high cost and effort of buying and managing their servers. Before AWS, businesses had to set up physical computers and storage to run websites or apps, which took time and money. AWS came into the market to solve this problem by offering these resources over the internet.



AWS and Its Core Capabilities

Amazon Web Services (AWS) is a cloud platform offered by Amazon that lets people and companies use IT services like storage, servers, and databases through the internet. Instead of buying and maintaining physical computers, you can "rent" these services online and only pay for what you use.

AWS offers over 200 services including:

EC2: Virtual servers for running applications.

S3: Object storage for files and media.

RDS: Managed relational databases.

Lambda: Run code without managing servers.

From startups to large enterprises like Netflix, Airbnb, and NASA, AWS is widely adopted for its flexibility, scalability, and security.

Understanding AWS Fundamentals

To effectively use AWS, you must understand its foundational concepts:

1. Regions

AWS operates in multiple geographical regions worldwide. Each region consists of one or more data centers and is designed to provide low-latency services to nearby users.

2. Availability Zones (AZs)

Each region includes multiple Availability Zones, which are isolated data centers. These zones provide high availability and fault tolerance, ensuring your application remains operational even if one zone fails.

3. Global Network Infrastructure

AWS has a global network of high-speed fiber connections linking its data centers. This infrastructure enables optimized performance, reduced latency, and better disaster recovery options.

How AWS Delivers Cloud Services

Amazon Web Services (AWS) delivers cloud computing through a network of globally distributed data centers connected by high-speed fiber. These centers host a wide range of services from storage and compute power to databases and machine learning that users can access on demand via the internet.

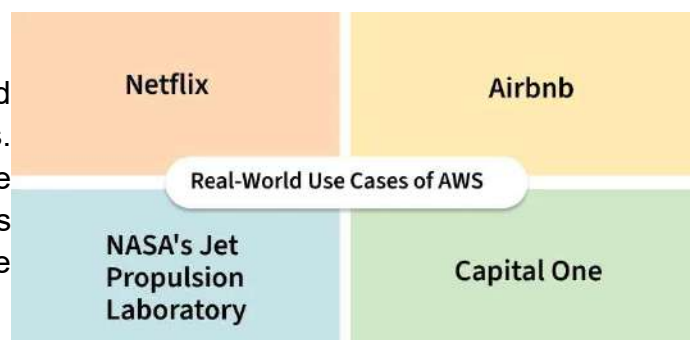
Whether you are a startup or an enterprise, AWS allows you to build and run applications without managing physical servers. AWS handles:

- Server maintenance Security patching
- Resource scaling
- High availability

For example, if you are running an e-Commerce site, instead of manually handling database setup, backups, and failover, you can use Amazon RDS. It automates all these backend tasks, enabling you to launch a database in minutes with just a few clicks.

Real-World Use Cases of AWS

AWS services are used by both startups and large enterprises based on their specific needs. Startups use AWS to overcome hardware infrastructure costs and deploy applications efficiently. Whereas large scale companies are



using AWS cloud services for the management of their Infrastructure to completely focus on the development of products widely.

Here are some real-world use cases of AWS services:

Netflix: The large streaming giant using AWS for the storage and scaling of the applications for ensuring seamless content delivery with low latency without interruptions to millions of users globally.

Airbnb: By utilizing AWS, Airbnb manages the various workloads and provides scalable and reliable infrastructure for its virtual marketplace and lodging offerings.

NASA's Jet Propulsion Laboratory: It takes the help of AWS services to handle and analyze large-scale volumes of data related to vital scientific research missions and space exploration.

Capital One: A financial Company that is utilizing AWS for its security and compliance while delivering innovative banking services to its customers.

AWS Global Infrastructure

The AWS global infrastructure is massive and is divided into geographical regions. The geographical regions are then divided into separate availability zones. While selecting the geographical regions for AWS, three factors come into play

- Optimising Latency
- Reducing cost
- Government regulations (Some services are not available for some regions)

Each region is divided into at least two availability zones that are physically isolated from each other, which ensures business continuity through distributed infrastructure. If one zone fails to function, the infrastructure in other availability zones remains operational. The largest region North Virginia (US-East), has six availability zones. These availability zones are connected by high-speed fiber-optic networking.

There are over 100 edge locations distributed all over the globe that are used for the Cloud-Front (content delivery network). Cloud-Front can cache frequently used content such as images and videos(including live streaming videos) at edge locations and distribute it to edge locations across the globe for high-speed delivery and low latency for end-users. It also protects from DDOS attacks.

AWS (Amazon Web Services) follows a pay-as-you-go pricing model, offering flexibility and scalability for businesses of all sizes. Pricing varies depending on the services you use, and AWS provides multiple options to help optimize costs. Here's an overview of key AWS pricing features and models:

1. Pay-as-You-Go Pricing

AWS charges for usage-based billing, meaning you only pay for what you use. This pricing model is based on factors like:

- Compute (e.g., EC2 instances)
- Storage (e.g., S3)

- Data transfer
- Requests and service usage (e.g., Lambda invocations)

This model is ideal for businesses with variable workloads.

2. On-Demand Instances

On-Demand Instances let you pay for compute capacity by the hour or second (depending on the instance type) with no long-term commitments or upfront payments. This option is ideal for:

- Applications with short-term, irregular, or unpredictable workloads
- First-time AWS users testing the platform
- Projects that cannot be interrupted

These instances offer maximum convenience and are perfect for development, testing, and prototyping workloads.

3. Reserved Pricing

For predictable usage, you can commit to a long-term contract (1 or 3 years) with reserved instances for services like EC2, RDS and Redshift,

- Up to 75% cost savings compared to on-demand pricing.
- Flexible payment options (All upfront, Partial upfront, or No upfront).



4. Spot Instances

Spot Instances allow you to bid on unused EC2 capacity. Prices fluctuate based on supply and demand, and you can save up to 90% compared to on-demand prices.

- Great for batch processing, data analysis, or flexible workloads.

5. Free Tier

AWS offers a Free Tier for new users, providing access to a limited set of services for free, such as:

- 750 hours/month of EC2 (t2.micro instance) for the first 12 months.
- 5GB of standard S3 storage.
- 1 million Lambda requests/month.

This is an excellent way for businesses to explore AWS without incurring costs.

To Know How to Set up Free Tier Account on Amazon Web Services

6. AWS Pricing Calculator

AWS provides a Pricing Calculator to estimate costs based on your specific usage. It helps you project the total cost of your cloud infrastructure by selecting services and configurations relevant to your business.

7. Cost Management and Optimization

AWS offers tools like AWS Cost Explorer and AWS Budgets to:

- Track usage and manage expenses.
- Set custom budgets and receive alerts when approaching limits.

Microsoft Windows Azure

WHAT IS MICROSOFT AZURE?

Microsoft Azure is a cloud computing platform that provides on-demand access to services like virtual machines, databases, AI tools, networking, storage, DevOps, and security. Think of it as renting powerful computing resources from Microsoft's data centers instead of owning your own hardware.

What can you do with Azure?

Compute

Virtual Machines (VMs)
App Services for hosting websites and APIs
Azure Functions (serverless computing)

Storage

Blob Storage (object storage)
File & Queue storage

AI & Machine Learning

Azure OpenAI Service
Cognitive Services (Vision, Speech, Language)

Databases

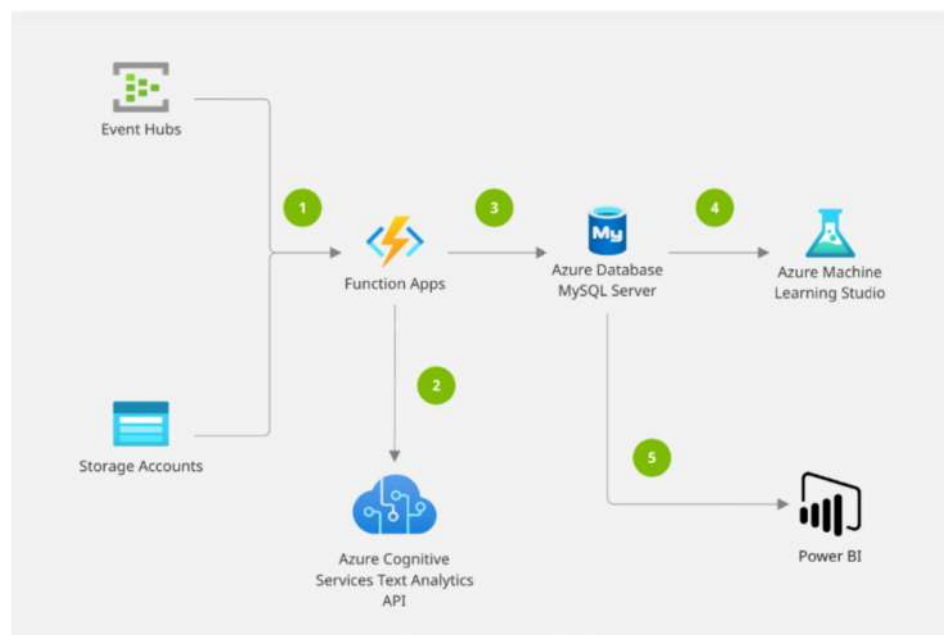
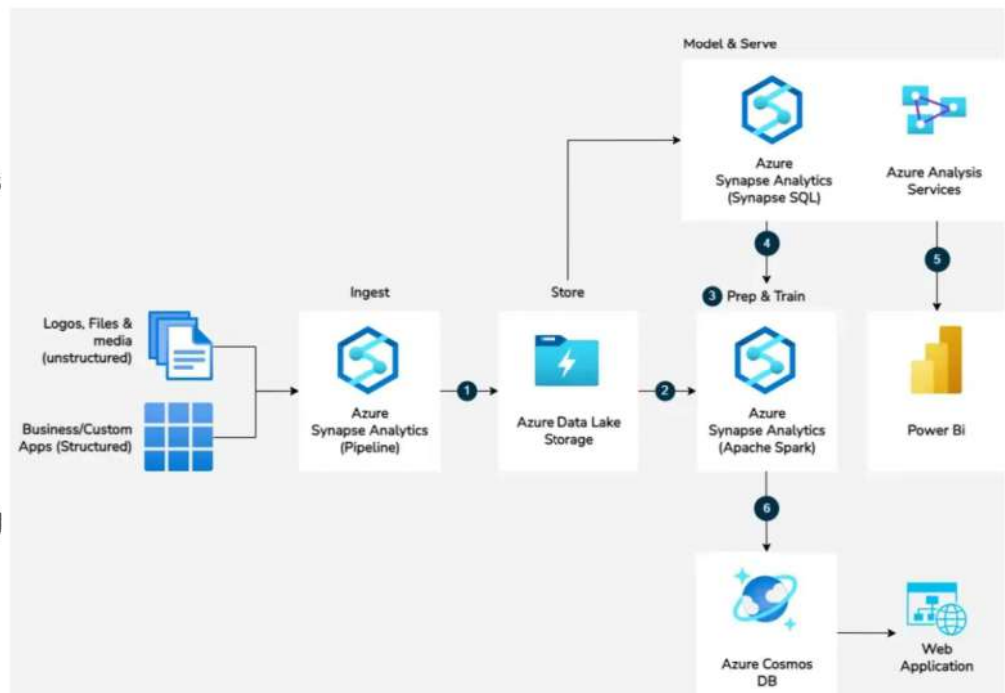
Azure SQL Database
Cosmos DB
MySQL/PostgreSQL managed services

Security

Azure Active Directory (Entra ID)
Key Vault
Security Center

DevOps & Integration

Azure DevOps
GitHub Actions
Logic Apps
API Management



Networking

Virtual Networks (Vnet), Load Balancers, VPN Gateways

Microsoft Azure is a cloud computing platform that provides a wide range of services for building, deploying, and managing applications. Formerly known as Windows Azure, it offers services like compute, storage, networking, and AI through a global network of Microsoft-managed data centers. This allows businesses to use cloud services instead of or alongside their own on-premises infrastructure.

Core functionalities:

Infrastructure as a Service (IaaS): Provides access to virtual machines, storage, and networking, giving you more control over the infrastructure.

Platform as a Service (PaaS): Offers a platform for developing and deploying applications without having to manage the underlying hardware or operating system.

Software as a Service (SaaS): Delivers software applications directly to end-users.

Server-less computing: Enables running applications without provisioning or managing servers.

Key services and features:

Compute:

Services for running applications and processing data.

Storage and databases:

Solutions for storing and managing data, including relational databases.

Networking:

Tools like Virtual Networks, VPNs, and load balancers to connect to on-premises data centers and manage traffic.

AI and machine learning:

Tools and services to build and deploy AI-powered solutions.

Security and identity:

Built-in security, identity management (now Microsoft Entra ID), and compliance services to protect data.

Hybrid and multi-cloud:

Capabilities to integrate and manage workloads across the cloud and on-premises environments

Benefits and Use Cases:

Organizations use Azure for benefits such as:

Scalability and Cost Efficiency:

The ability to dynamically adjust resources based on demand and the PAYG model helps manage costs effectively.

Hybrid Cloud Capabilities:

Seamlessly extends on-premises infrastructure to the cloud, allowing for operational consistency across environments.

Global Reach:

A vast network of data centers across more than 60 regions ensures high availability and reliable performance for applications worldwide.

Security and Compliance:

Built-in security solutions and adherence to a wide range of compliance standards help protect data and meet regulatory requirements

Service Oriented Architecture: REST

What is Service-Oriented Architecture (SOA)?

SOA (Service-Oriented Architecture) is a design style where applications are built as a collection of independent services that communicate over a network.

Each service:

- Performs a specific business function
- Communicates via well-defined interfaces
- Can be reused across applications

Typical SOA Characteristics

- Services communicate through standard protocols
- Loose coupling
- Reusability & interoperability
- Often used in large enterprise systems

Traditional SOA implementations often used:

- SOAP (Simple Object Access Protocol)
- WSDL (Web Services Description Language)



WHAT IS REST?

REST (Representational State Transfer) is an architectural style for building web services using simple **HTTP operations**

REST uses:

- HTTP methods → GET, POST, PUT, DELETE
- Resources identified by URLs
- Standard formats → JSON, XML (JSON preferred)

Core REST principles:

- Statelessness – No session stored on server
- Client–Server separation
- Uniform interface
- Cacheable responses

- Layered system

SOA VS REST — QUICK COMPARISON

Aspect	SOA	REST
Protocol	Heavy protocols like SOAP	Simple HTTP
Data format	XML (mainly)	JSON / XML
Messaging	Can be complex (WS-*)	Lightweight
Best for	Enterprise systems, transactions	Web/mobile apps, microservices
Coupling	More tightly coupled	Loosely couple

HOW REST FITS INTO SOA

REST is **not separate from SOA** — REST **CAN BE USED INSIDE AN SOA** as a way to expose services.

SOA = architectural style for structuring services

REST = architectural style for communication between services

Today, modern systems use:

- **Microservices architecture**
- REST APIs for communication
- Or **gRPC / messaging systems** for faster, scalable interaction

Service-Oriented Architecture (SOA) is an architectural design philosophy that structures applications as a collection of independent, reusable, and interoperable services. **REST** (Representational State Transfer) is an architectural style for designing networked applications, and it is a common, lightweight technology used to implement SOA, particularly in modern cloud-based environments and public APIs.

Service-Oriented Architecture (SOA)

SOA promotes the reuse of business functionalities as services with well-defined interfaces.

•Key Principles:

- **Loose Coupling:** Services are designed to be self-contained with minimal dependencies on other services, so one can be updated without affecting others.
- **Interoperability:** Services can communicate across different platforms and programming languages using standardized protocols.

- **Abstraction:** The internal logic and implementation details of a service are hidden from its consumers.
- **Reusability:** Services are intended to be used as building blocks across multiple applications and business processes.
- **Statelessness (encouraged):** Services typically minimize retaining information from past sessions, which improves scalability.
- **Discoverability:** Services are often published in a service registry or repository, allowing developers to find and reuse them.
- **Composability:** Services can be combined to form more complex operations or composite services.
- **Communication:** SOA services communicate using established protocols. Traditionally, SOA has been associated with heavyweight, XML-based protocols like SOAP (Simple Object Access Protocol) and WSDL (Web Services Description Language), often managed through a centralized Enterprise Service Bus (ESB).

REST in the Context of SOA

REST is a specific architectural style that aligns well with the principles of SOA and has largely become the preferred implementation method for modern services, especially microservices.

- **Architectural Style vs. Protocol:** It's important to note that REST is an *architectural style* with a set of constraints, while SOAP is a *protocol*.
- **Lightweight Communication:** RESTful services typically use standard, lightweight HTTP methods (like GET, POST, PUT, DELETE) and exchange data in formats such as JSON or XML.
- **Statelessness:** A core constraint of REST is statelessness, meaning each request must contain all necessary information for the server to understand and process it, without relying on prior interactions. This inherent statelessness makes REST highly scalable and efficient for web-based applications.
- **Simplicity and Performance:** REST APIs are generally simpler to understand, learn, and implement compared to the more formal and complex SOAP/WSDL model. Their smaller message sizes also contribute to faster performance.

Summary of Relationship

REST is a modern, highly effective way to build services that adhere to the principles of SOA. While traditional SOA implementations heavily relied on SOAP and ESBs, REST provides a lighter, more flexible, and more scalable alternative that is well-suited for current cloud and mobile computing environments

Publish Subscribe Model

Publish–Subscribe Model (Pub/Sub)

The **Publish–Subscribe model** is a messaging pattern where **senders (publishers)** do not communicate directly with **receivers (subscribers)**. Instead, they interact through a **message broker**.

The Publish–Subscribe (Pub/Sub) model is a messaging pattern used in distributed systems to enable asynchronous, event-driven communication between components. It helps decouple systems so that publishers and subscribers do not need to know about each other directly.

Publish–Subscribe (Pub/Sub) Model

The Publish–Subscribe model is a messaging pattern used in distributed systems to enable asynchronous, event-driven communication between components.

Key Components:

1. Publisher – Sends messages/events without knowing who receives them.
2. Subscriber – Listens to only the topics it is interested in.
3. Message Broker – Routes messages from publishers to subscribers.

How It Works:

1. Publishers send messages to a topic.
2. Broker receives the message.
3. Broker delivers the message to all subscribers of that topic.

Advantages:

- Loose coupling
- Scalability
- Asynchronous communication
- Multiple receivers for a single event

Common Use Cases:

- Event-driven architectures
- IoT systems
- Real-time analytics

- Microservices communication

Popular Pub/Sub Technologies:

Kafka, RabbitMQ, Redis Pub/Sub, Google Pub/Sub, AWS SNS, Azure Event Hub, MQTT brokers.

The **Publish/Subscribe (Pub/Sub) model** is an asynchronous messaging pattern where data producers (publishers) broadcast messages to an intermediary (a message broker) without needing to know the specific recipients. Interested consumers (subscribers) register their interest in specific **topics** or channels and receive all messages published to those topics.

This pattern enables scalable and loosely coupled communication between different parts of a system.

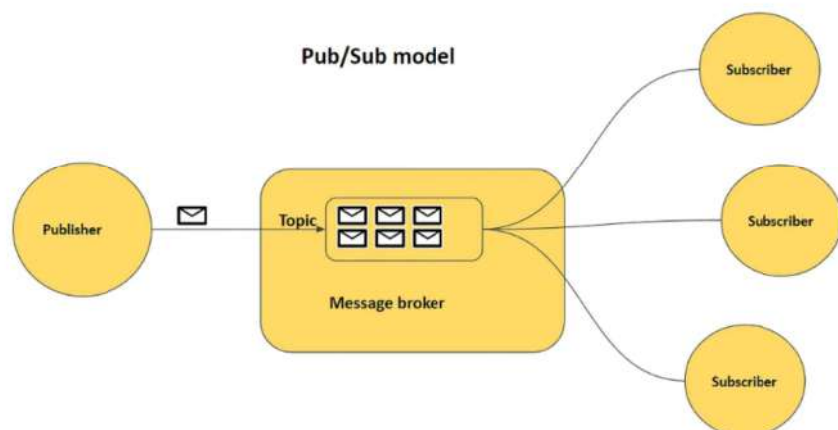
Key Components

The Pub/Sub architecture consists of four main components:

- Publisher:** The component that creates and sends messages (events) to the messaging system. The publisher operates independently and is unaware of which, or how many, subscribers exist.
- Subscriber:** The component that expresses interest in specific topics and receives relevant messages from the system. Subscribers can perform different functions with the same message in parallel.
- Topic/Channel:** A named feed or category that acts as an intermediary channel for messages. Publishers send messages to topics, and subscribers subscribe to them.
- Message Broker:** The core infrastructure component that manages, stores, and routes messages from publishers to the appropriate subscribers based on their subscriptions.

How It Works

1. A publisher creates a message and sends it to a specific topic on the message broker.
2. The message broker receives the message and identifies all subscribers interested in that topic.
3. The broker then reliably pushes the message to all identified subscribers asynchronously.



4.Subscribers process the message independently at their own pace, without the publisher needing to wait for a response

Benefits

- Loose Coupling:** Publishers and subscribers are decoupled both spatially (they don't need to know each other's location or identity) and temporally (they don't need to be running at the same time).
- Scalability:** The system can scale dynamically. You can add more publishers or subscribers without disrupting existing components, as message distribution is managed by the broker.
- Asynchronous Communication:** Non-blocking message exchange improves responsiveness and overall system efficiency.
- Reliability:** Many systems provide mechanisms for message durability, retries, and acknowledgments to ensure messages aren't lost due to temporary failures.

Common Use Cases

The Pub/Sub model is widely used in modern software architecture.

- Real-time Applications:** Used for live updates in chat applications, online gaming, and stock trading platforms.
- IoT (Internet of Things):** Enables seamless communication between a multitude of devices and backend systems, such as smart home automation or remote patient monitoring.
- Event-Driven Architectures:** A foundational pattern for microservices that need to react to events without direct dependencies.
- Notifications and Alerts:** Instantly delivers critical updates and notifications to users or other application components (e.g., social media feeds).
- Logging and Monitoring:** Centralized systems can aggregate logs from many services for analysis

Technologies

Various technologies and cloud services implement the Pub/Sub pattern, including:

- Amazon Simple Notification Service (SNS)
- [Google Cloud Pub/Sub](#)
- [Apache Kafka](#)
- [RabbitMQ](#), MQTT, 43zA a lightweight protocol often used for IoT